S.No: 1	Exp. Name: ***Program to implement AVL tree and its operations***	**Date: 2026-09-13**

ID: 25ME1A5483

Ramachandra College of Engineering

Aim:

Write a C program to create an AVL tree using a provided set of elements stored in a file. Implement operations for inserting and deleting elements from the AVL tree. After constructing the tree, output the contents of the tree in in-order traversal to a new file.

Input and Output Format:

- The program first prompts the user: "Input file name: ". If the file cannot be opened, display: "Error opening input file".
- If the file opens successfully:
  - First line: integer n (number of elements)
  - Next n lines: integers to be inserted into the AVL tree.
- Display:

```
1. Insert
2. Delete
3. Write to file and exit
Choice:
```

- If choice is 1, prompt: "Element: " and insert the element into the AVL tree. After insertion, display: "In-order traversal: " followed by space-separated values.
- If choice is 2, prompt: "Element: " and attempt to delete it from the AVL tree.
  - If the element is found and deleted, display: "Deleted <element>"
  - If the element is not found, display: "<element> not found"
  - After the deletion attempt, display: "In-order traversal: " followed by space-separated values.
- If choice is 3, prompt: Output file name: and write the in-order traversal to that file
  - If the output file is opened and written successfully, display: "AVL Tree written to file".
  - If the output file cannot be opened, display: "Error opening output file".
  - If the user enters an invalid choice, display: "Invalid choice".

Source Code:

avl_tree.c

```
// Write your code here...
#include <stdio.h>
#include <stdlib.h>

// Define structure for an AVL tree node
struct Node
{
int key;
struct Node* left;
struct Node* right;
int height;
};

struct DeletionResult
{
struct Node* root;
int deleted;// 1 if deleted, 0 if not found
};

// Function to get the height of the tree
int height(struct Node* N){
        if(N == NULL)
                return 0;
        return N -> height;
}

// Function to get the maximum of two integers
int max(int a, int b){
        return (a > b) ? a : b;
}

// Function to allocate a new node
struct Node* newNode(int key){
        struct Node* node = (struct Node*)malloc(sizeof(struct Node));
        node -> key = key;
        node -> left = NULL;
        node -> right = NULL;
        node -> height = 1; // New node is initially added at leaf
        return node;
}

// Right rotate subtree rooted with y
struct Node* rightRotate(struct Node* y){
        struct Node* x = y -> left;
```

```
        struct Node* T2 = x -> right;

        // Perform rotation

        x -> right = y;
        y -> left = T2;

        // Update heights
        y -> height = max(height(y -> left), height(y -> right)) + 1;
        x -> height = max(height(x -> left), height(x -> right)) + 1;

        return x;
}

// Left rotate subtree rooted with x
struct Node* leftRotate(struct Node* x){
        struct Node* y = x -> right;
        struct Node* T2 = y -> left;

        // Perform rotation

        y -> left = x;
        x -> right = T2;

        // Update heights
        x -> height = max(height(x -> left), height(x -> right)) + 1;
        y -> height = max(height(y -> left), height(y -> right)) + 1;

        return y;
}

// Get balance factor of node N
int getBalance(struct Node* N){
        if(N == NULL)
                return 0;
        return height(N -> left) - height(N -> right);
}

// Recursive function to insert a key in the subtree rooted with node
and return the new root
struct Node* insert(struct Node* node, int key){

        // normal BST insertion
        if(node == NULL)
                return newNode(key);
        if(key < node -> key)
```

```
                node -> left = insert(node -> left,key);
        else if(key > node -> key)
                node -> right = insert(node -> right,key);
        else
                return node;

        // update height
        node -> height = 1 + max(height(node -> left),height(node ->
right));

        // get balance factor
        int balance = getBalance(node);

        // LL case
        if(balance > 1 && key < node -> left -> key)
                return rightRotate(node);

        // RR case
        if(balance < -1 && key > node -> right -> key)
                return leftRotate(node);

        // LR case
        if(balance > 1 && key > node -> left -> key){
                node -> left = leftRotate(node -> left);
                return rightRotate(node);
        }

        // RL case
        if(balance < -1 && key < node -> right -> key){
                node -> right = rightRotate(node -> right);
                return leftRotate(node);
        }

        return node;
}

// Function to find the node with minimum value in the tree
struct Node* minValueNode(struct Node* node){
        struct Node* current = node;

        // Find the leftmost leaf
        while (current -> left != NULL)
                current = current -> left;

        return current;
}
```

```
// Recursive function to delete a node with a given key from subtree
with given root
struct DeletionResult deleteNode(struct Node* root, int key){
        struct DeletionResult result;
        result.deleted = 0;
        result.root = root;

        // key not found
        if(root == NULL){
                result.root = NULL;
                return result;
        }

        // search left subtree
        if(key < root -> key){
                struct DeletionResult temp = deleteNode(root ->
left,key);
                root -> left = temp.root;
                result.deleted = temp.deleted;
        }

        // search right subtree
        else if(key > root -> key){
                struct DeletionResult temp = deleteNode(root ->
right,key);
                root -> right = temp.root;
                result.deleted = temp.deleted;
        }

        // node found
        else{
                result.deleted = 1;
                // node with no child or one child
                if(root -> left == NULL || root -> right == NULL){
                        struct Node* temp;
                        if(root -> left != NULL)
                                temp = root -> left;
                        else
                                temp = root -> right;
                        // no child
                        if(temp == NULL){
                                free(root);
                                result.root = NULL;
                                return result;
                        }
```

```
                        // one child
                        else{
                                struct Node* oldRoot = root;
                                root = temp;
                                free(oldRoot);
                        }
                }
                // node with two children
                else{
                        struct Node* temp = minValueNode(root -> right);
                        root -> key = temp->key;
                        struct DeletionResult tempResult =
deleteNode(root -> right,temp -> key);
                        root -> right = tempResult.root;
                }
        }
        // if tree is empty
        if(root == NULL){
                result.root = NULL;
                return result;
        }
        // update height
        root -> height = 1 + max(height(root -> left),height(root ->
right));

        // get balance factor
        int balance = getBalance(root);

        // LL case
        if(balance > 1 && getBalance(root -> left) >= 0){
                root = rightRotate(root);
        }

        // RR case
        else if(balance < -1 && getBalance(root -> right) <= 0){
                root = leftRotate(root);
        }

        // LR case
        else if(balance > 1 && getBalance(root -> left) < 0){
                root -> left = leftRotate(root -> left);
                root = rightRotate(root);
        }

        //RL case
        else if(balance < -1 && getBalance(root -> right) > 0){
```

```
                root -> right = rightRotate(root -> right);
                root = leftRotate(root);
        }

        result.root = root;
        return result;
}

// Function to perform in-order traversal
void inOrderTraversal(struct Node* root){

        if(root == NULL)
                return;
        inOrderTraversal(root -> left);
        printf("%d ",root -> key);
        inOrderTraversal(root -> right);
}

// Function to write in-order traversal to a file
void writeToFile(struct Node* root, FILE* file){
        if(root != NULL){
                writeToFile(root -> left, file);
                fprintf(file, "%d ",root -> key);
                writeToFile(root -> right, file);
        }
}

int main(){
        char inputFileName[100], outputFileName[100];
        int choice, element;

        printf("Input file name: ");
        scanf("%s",inputFileName);

        FILE* inputFile = fopen(inputFileName,"r");

        if(inputFile == NULL){
                printf("Error opening input file\n");
                return 1;
        }
        int n;
        struct Node* root = NULL;

        // Read the number of elements and insert into AVL tree
        fscanf(inputFile, "%d", &n);
        for (int i = 0;i < n;i++){
```

```
                fscanf(inputFile, "%d", &element);
                root = insert(root, element);
        }
        printf("In-order traversal:\n");
        inOrderTraversal(root);

        fclose(inputFile);

        while(1){
                printf("\n1. Insert\n2. Delete\n3. Write to file and
exit\nChoice: ");
                scanf("%d", &choice);

                switch(choice){

                        case 1:
                        printf("Element: ");
                        scanf("%d", &element);
                        root = insert(root, element);
                        printf("In-order traversal:\n");
                        inOrderTraversal(root);
                        break;

                        case 2: // Deleting an element
                        printf("Element: ");
                        scanf("%d", &element);
                        struct DeletionResult deleteResult =
deleteNode(root,element);
                        root = deleteResult.root;
                        if(deleteResult.deleted){
                                printf("Deleted %d\n", element);
                        }else{
                                printf("%d not found\n",element);
                        }
                        printf("In-order traversal:\n");
                        inOrderTraversal(root);
                        break;

                        case 3:
                        printf("Output file name: ");
                        scanf("%s", outputFileName);
                        FILE* outputFile = fopen(outputFileName, "w");
                        if(outputFile == NULL) {
                                printf("Error opening output file");
                                return 1;
                        }
```

```
                    writeToFile(root, outputFile);
                    fclose(outputFile);
                    printf("AVL Tree written to file");
                    return 0;

                    default:
                    printf("Invalid choice");
                    break;
                }
            }
            return 0;
}
```

input1.txt

```
5
1 2 3 4 5
```

input2.txt

```
6
12 24 36 48 60 72
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Input file name:
inputfile1.txt
Error opening input file

Test Case - 2
User Output
Input file name:
input1.txt
In-order traversal:

1 2 3 4 5
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
10
In-order traversal:
1 2 3 4 5 10
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
100
100 not found
In-order traversal:
1 2 3 4 5 10
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
3
Deleted 3
In-order traversal:
1 2 4 5 10
1. Insert
2. Delete
3. Write to file and exit
Choice:
3
Output file name:
myoutput.txt
AVL Tree written to file

Test Case - 3
User Output

Input file name:
input2.txt
In-order traversal:
12 24 36 48 60 72
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
108
In-order traversal:
12 24 36 48 60 72 108
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
120
In-order traversal:
12 24 36 48 60 72 108 120
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
144
In-order traversal:
12 24 36 48 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
36
Deleted 36
In-order traversal:
12 24 48 60 72 108 120 144
1. Insert

```
2. Delete
3. Write to file and exit
Choice:
2
Element:
48
Deleted 48
In-order traversal:
12 24 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
10
Invalid choice
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
1440
1440 not found
In-order traversal:
12 24 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
15
Invalid choice
1. Insert
2. Delete
3. Write to file and exit
Choice:
3
Output file name:
outputfile.txt
AVL Tree written to file
```

S.No: 2	Exp. Name: ***Operation of B-Tree - Insertion, Traversal***	**Date: 2026-09-16**

Aim:

Write a C program to implement a B-Tree and perform insertion and traversal operations on it. The program should allow the user to insert elements into the B-Tree while maintaining the properties of a B-Tree. Duplicate elements should not be inserted. The contents of the B-Tree should be displayed using in-order traversal.

Input and Output Format:

- Display the following menu:

```
1.Insert 2.Traversal 3.Exit
Enter your option :
```

- If the user enters 1, prompt:

```
Enter an element to be inserted :
```

Insert the given element into the B-Tree.

- If the element is already present in the B-Tree, display:

```
Duplicates not allowed
```

- If the user enters 2:
- If the B-Tree is empty, display:

```
B-Tree is empty
```

- Otherwise, display the elements of the B-Tree using in-order traversal in the following format:

```
Elements of the B-Tree : <space-separated elements>
```

- If the user enters 3, terminate the program.
- If the user enters any other option, no operation needs to be performed.
- The program should continue accepting operations until the user chooses option 3.

Note:

- The B-Tree should have a maximum of **4 keys per node**.
- Maintain the B-Tree properties after every insertion.
- Duplicate elements are not allowed.
- The traversal should display the elements in **sorted order**.

Source Code:

BTreeOperations1.c

```
// Write your code here...
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

#define MAX 4
#define MIN 2

struct BTreeNode {
int val[MAX + 1], count;
struct BTreeNode *link[MAX + 1];
};

typedef struct BTreeNode *BTNODE;

BTNODE root;

// creating new node
BTNODE createNode(int val,BTNODE child){
        BTNODE newNode = (BTNODE)malloc(sizeof(struct BTreeNode));

        newNode -> val[1] = val;
        newNode -> count = 1;

        newNode -> link[0] = root;
        newNode -> link[1] = child;

        return newNode;
}

// Fill the code in below functions. please refer to the psuedo codde.
void addValToNode(int val, int pos, BTNODE node, BTNODE child){
        int j;
        for(j = node -> count;j > pos;j--){
                node -> val[j + 1] =node -> val[j];
                node -> link[j + 1] =node -> link[j];
        }

        node -> val[j + 1] = val;
        node -> link[j + 1] = child;
        node -> count++;
}

void splitNode(int val, int *pval, int pos, BTNODE node, BTNODE child,
```

```
BTNODE *newNode){
        int median,j;

        if(pos > MIN)
                median = MIN +1;
        else
                median = MIN;

        *newNode = (BTNODE)malloc(sizeof(struct BTreeNode));

        j = median + 1;

        while(j == MAX){
                (*newNode) -> val[j - median] = node -> val[j];
                (*newNode) -> link[j - median] = node -> link[j];
                j++;
        }

        (*newNode) -> count = MAX - median;
        node -> count = median;

        if(pos <= MIN){
                addValToNode(val,pos,node,child);
        }else{
                addValToNode(val,pos - median,*newNode,child);
        }

        *pval = node -> val[node -> count];
        (*newNode) -> link[0] = node -> link[node -> count];
        node -> count--;
}

int setValueInNode(int val, int *pval, BTNODE node, BTNODE* child){
        int pos;
        if(! node){
                *pval = val;
                *child = NULL;
                return 1;
        }
        if(val < node -> val[1]){
                pos = 0;
        }else{
                for(pos = node -> count;(pos > 1 && val < node ->
val[pos]);pos--);
                if(val == node -> val[pos]){
                        printf("Duplicates not allowed\n");
```

```
                        return 0;
                }
        }
        if(setValueInNode(val,pval,node -> link[pos],child)){
                if(node -> count < MAX){
                        addValToNode(*pval,pos,node,*child);
                }else{
                        splitNode(*pval,pval,pos,node,*child,child);
                        return 1;
                }
        }
        return 0;
}

void insertNodeInBTree(int val){
        int flag, i;
        BTNODE child;
        flag = setValueInNode(val,&i,root,&child);
        if(flag)
                root = createNode(i,child);
}

void traverseBTree(BTNODE myNode) {
        int i;
        if(myNode){
                for(i = 0;i < myNode -> count;i++){
                        traverseBTree(myNode -> link[i]);
                        printf("%d ",myNode -> val[i + 1]);
                }
                traverseBTree(myNode -> link[i]);
        }
}

int main(){
        int choice,val;

        root = NULL;

        while(1){
                printf("1.Insert 2.Traversal 3.Exit\n");

                printf("Enter your option : ");
                scanf("%d",&choice);

                switch(choice){
                        case 1:
```

```
                        printf("Enter an element to be inserted : ");
                        scanf("%d",&val);
                        insertNodeInBTree(val);
                        break;

                        case 2:
                        if(root == NULL)
                                printf("B-Tree is empty\n");
                        else{
                                printf("Elements of the B-Tree : ");
                                traverseBTree(root);
                                printf("\n");
                        }
                        break;

                        case 3:
                        exit(0);

                        default:
                        printf("\nInvalid choice");
                }
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1.Insert 2.Traversal 3.Exit
Enter your option :
1
Enter an element to be inserted :
5
1.Insert 2.Traversal 3.Exit
Enter your option :
1
Enter an element to be inserted :
10
1.Insert 2.Traversal 3.Exit

Enter your option :
1
Enter an element to be inserted :
10
Duplicates not allowed
1.Insert 2.Traversal 3.Exit
Enter your option :
2
Elements of the B-Tree : 5 10
1.Insert 2.Traversal 3.Exit
Enter your option :
3

Test Case - 2
User Output
1.Insert 2.Traversal 3.Exit
Enter your option :
2
B-Tree is empty
1.Insert 2.Traversal 3.Exit
Enter your option :
1
Enter an element to be inserted :
9
1.Insert 2.Traversal 3.Exit
Enter your option :
1
Enter an element to be inserted :
15
1.Insert 2.Traversal 3.Exit
Enter your option :
2
Elements of the B-Tree : 9 15
1.Insert 2.Traversal 3.Exit
Enter your option :
3

S.No: 3	Exp. Name: ***Operation of B-Tree: Deletion, Search***	**Date: 2026-09-27**

Aim:

Write a C program to implement a B-Tree and perform Insertion, Deletion, Search, and Traversal operations on it. The program should maintain the properties of the B-Tree after every insertion and deletion. Duplicate elements should not be inserted into the B-Tree.

Input and Output Format:

- Display the following menu:

```
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
```

- If the user enters 1, prompt:

```
Enter an element to be inserted :
```

Insert the given element into the B-Tree.

- If the element is already present in the B-Tree, display:

```
Duplicates not allowed.
```

- If the user enters 2, prompt:

```
Enter the element to be deleted :
```

Attempt to delete the given element from the B-Tree.

- If the element is successfully deleted, no additional message needs to be displayed.
- If the element is not present in the B-Tree, display:

```
Element not found in B-Tree.
```

- If the user enters 3, prompt:

```
Enter the element to be searched :
```

Search for the given element in the B-Tree.

- If the element is found, display:

```
Element found in B-Tree
```

- If the element is not found, display:

```
Element not found in B-Tree
```

- If the user enters 4:
- If the B-Tree is empty, display:

```
B-Tree is empty
```

- Otherwise, display the elements of the B-Tree in sorted order using traversal in the following format:

```
Elements of the B-Tree : <space-separated elements>
```

- If the user enters 5, terminate the program.
- The program should continue accepting operations until the user chooses option 5.

Note:

- The B-Tree should have a maximum of **4 keys per node**.
- Maintain the properties of the B-Tree after every insertion and deletion.
- Duplicate elements are not allowed.

• The traversal operation should display the elements in **sorted order**.

Source Code:

BTreeOperations2.c

```
// Write your code here...
#include <stdio.h>
#include <stdlib.h>

#define MAX 4

struct Node{
int key[MAX];
struct Node *child[MAX + 1];
int n;
int leaf;
};

struct Node *root = NULL;
int firstElement = 1;

// Create Node
struct Node *createNode(int leaf){
        struct Node *p;
        int i;

        p = (struct Node*)malloc(sizeof(struct Node));

        p -> n = 0;
        p -> leaf = leaf;

        for(i = 0;i <= MAX;i++)
                p -> child[i] = NULL;

        return p;
}

//search
int search(struct Node *p,int x){
        int i = 0;
        if(p == NULL)
                return 0;
        while(i < p -> n && x > p -> key[i])
                i++;
        if(i < p -> n && x == p -> key[i])
                return 1;
        if(p -> leaf)
                return 0;
```

```
        return search(p -> child[i],x);
}

//split node
void split(struct Node *parent,int pos){
        struct Node *a;
        struct Node *b;
        int i;
        int middle = 2;

        a = parent -> child[pos];
        b = createNode(a -> leaf);

        //copy keys after middle
        b -> n = a -> n - middle - 1;
        for(i = 0;i < b -> n;i++)
                b -> key[i] = a -> key[middle + 1 + i];
        //copy child pointers if internal node
        if(!a -> leaf){
                for(i = 0;i <= b -> n;i++)
                        b -> child[i] = a -> child[middle + 1 +i];
        }
        //reduce left node
        a -> n = middle;
        //shift children
        for(i = parent -> n;i >= pos + 1;i--)
                parent -> child[i + 1] = parent -> child[i];
        parent -> child[pos + 1] = b;
        //shift keys
        for(i = parent -> n-1;i >= pos;i--)
                parent -> key[i + 1] = parent -> key[i];
        //move middle key to parent
        parent -> key[pos] = a -> key[middle];
        parent -> n++;
}

//insert into non-full node
void insertNonFull(struct Node *p,int x){
        int i;
        i = p -> n-1;
        if(p -> leaf){
                while(i >= 0 && x < p -> key[i]){
                        p -> key[i + 1] = p -> key[i];
                        i--;
                }
                p -> key[i + 1] = x;
```

```
                p -> n++;
        }
        else{
                while(i >= 0 && x < p -> key[i])
                        i--;
                i++;
                if(p -> child[i] -> n == MAX){
                        split(p,i);
                        if(x > p -> key[i])
                                i++;
                }
                insertNonFull(p -> child[i],x);
        }
}

//insert
void insert(int x){
        struct Node *newRoot;
        if(search(root,x)){
                printf("Duplicates not allowed.\n");
                return;
        }
        if(root == NULL){
                root = createNode(1);
                root -> key[0] = x;
                root -> n = 1;
                return;
        }
        if(root -> n == MAX){
                newRoot = createNode(0);
                newRoot -> child[0] = root;
                root = newRoot;
                split(root,0);
                insertNonFull(root,x);
        }
        else{
                insertNonFull(root,x);
        }
}

//find minimum
int findMin(struct Node *p){
        while(!p -> leaf)
                p = p -> child[0];
        return p -> key[0];
}
```

```
//find maximum
int findMax(struct Node *p){
        while(!p -> leaf)
                p = p -> child[p -> n];
        return p -> key[ p -> n-1];
}

//borrow key from previous child
void borrowFromPrev(struct Node *p,int idx){
        struct Node *child = p -> child[idx];
        struct Node *sibling = p -> child[idx - 1];

        int i;

        // shift child's keys
        for(i = child -> n - 1;i >= 0;i--)
                child -> key[i + 1] = child -> key[i];
        //move parent key to child
        child -> key[0] = p -> key[idx - 1];
        //move sibling's last child
        if(!child -> leaf){
                for(i = child -> n;i >= 0;i--)
                        child -> child[i + 1] = child -> child[i];
                child -> child[0] = sibling -> child[sibling -> n];
        }
        //move sibling key to parent
        p -> key[idx - 1] = sibling -> key[sibling -> n-1];
        child -> n++;
        sibling -> n--;
}

//borrow key from next child
void borrowFromNext(struct Node *p,int idx){
        struct Node *child = p -> child[idx];
        struct Node *sibling = p -> child[idx + 1];

        int i;

        //move parent key to child
        child -> key[child -> n] = p -> key[idx];
        //move sibling's first child
        if(!child -> leaf)
                child -> child[child -> n+1] = sibling -> child[0];
        //move sibling's first key to parent
        p -> key[idx] = sibling -> key[0];
```

```
        //shift sibling keys
        for(i = 1;i < sibling -> n;i++)
                sibling -> key[i-1] = sibling -> key[i];
        //shift sibling children
        if(!sibling -> leaf){
                for(i = 1;i <= sibling -> n;i++)
                        sibling -> child[i -1] = sibling -> child[i];
        }
        child -> n++;
        sibling -> n--;
}

//merge child with next sibling
void merge(struct Node *p,int idx){
        struct Node *child = p -> child[idx];
        struct Node *sibling = p -> child[idx + 1];

        int i;

        //move parent key into child
        child -> key[child -> n] = p -> key[idx];
        child -> n++;
        //copy sibling keys
        for(i = 0;i< sibling -> n;i++){
                child -> key[child -> n] = sibling -> key[i];
                child -> n++;
        }
        //copy sibling children
        if(!child -> leaf){
                for(i = 0;i <= sibling -> n;i++)
                        child -> child[child -> n - sibling -> n+i] =
sibling -> child[i];
        }
        //shift parent keys
        for(i = idx + 1;i < p -> n;i++)
                p -> key[i-1] = p -> key[i];
        //shift parent children
        for(i = idx + 2;i <= p -> n;i++)
                p -> child[i -1] = p -> child[i];

        p -> n--;

        free(sibling);
}

//fill child if it has minimum keys
```

```
void fill(struct Node *p,int idx){
        if(idx != 0 && p -> child[idx - 1] -> n >1){
                borrowFromPrev(p,idx);
        }
        else if(idx != p -> n && p -> child[idx + 1] -> n >1){
                borrowFromNext(p,idx);
        }
        else{
                if(idx != p -> n)
                        merge(p,idx);
                else
                        merge(p,idx-1);
        }
}

//delete key
void deleteKey(struct Node *p,int x){
        int idx = 0;
        int i;
        while(idx < p -> n && x > p ->key[idx])
                idx++;
        //key found
        if(idx < p -> n && p -> key[idx] == x){
                //case 1: leaf
                if(p -> leaf){
                        for(i = idx + 1;i < p -> n;i++)
                                p  -> key[i-1] = p -> key[i];
                        p -> n--;
                }
                else{
                //case 2: internal node
                        if(p -> child[idx] -> n > 1){
                                int pred = findMax(p -> child[idx]);
                                p -> key[idx] = pred;
                                deleteKey(p -> child[idx],pred);
                        }
                        else if(p -> child[idx + 1] -> n > 1){
                                int succ = findMin(p -> child[idx + 1]);
                                p -> key[idx] = succ;
                                deleteKey(p -> child[idx + 1],succ);
                        }
                        else{
                                merge(p,idx);
                                deleteKey(p -> child[idx],x);
                        }
                }
```

```
                return;
        }
        //key not found in current node
        if(p -> leaf)
                return;
        //make sure child has enough keys
        if(p -> child[idx] -> n == 1)
                fill(p,idx);
        //after merge,index may change
        if(idx > p -> n)
                deleteKey(p -> child[idx - 1],x);
        else
                deleteKey(p -> child[idx],x);
}

//delete element
int deleteElement(int x){
        struct Node *temp;
        if(!search(root,x))
                return 0;
        deleteKey(root,x);
        //if root becomes empty
        if(root != NULL && root -> n == 0){
                temp = root;
                if(root -> leaf)
                        root = NULL;
                else
                        root = root -> child[0];
                free(temp);
        }
        return 1;
}

//traversal
void traversal(struct Node *p){
        int i;
        if(p == NULL)
                return;
        for(i = 0;i < p -> n;i++){
                if(!p -> leaf)
                        traversal(p -> child[i]);
                printf("%d ",p -> key[i]);
        }
        if(!p -> leaf)
                traversal(p -> child[p -> n]);
}
```

```
// main function
int main(){
        int option;
        int x;
        while(1){
                printf("1.Insert 2.Delete 3.Search 4.Traversal
5.Exit\n");
                printf("Enter your option : ");
                scanf("%d",&option);

                switch(option){
                        case 1:
                        printf("Enter an element to be inserted : ");
                        scanf("%d",&x);
                        insert(x);
                        break;
                        case 2:
                        printf("Enter the element to be deleted : ");
                        scanf("%d", &x);
                        if(!deleteElement(x))
                                printf("Element not found in B-
Tree.\n");
                        break;
                        case 3:
                        printf("Enter the element to be searched : ");
                        scanf("%d",&x);
                        if(search(root,x))
                                printf("Element found in B-Tree\n");
                        else
                                printf("Element not found in B-Tree\n");
                        break;
                        case 4:
                        if(root == NULL)
                                printf("B-Tree is empty");
                        else
                                printf("Elements of the B-Tree : ");
                        traversal(root);
                        printf("\n");
                        break;
                        case 5:
                        return 0;
                }
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
3
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
4
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
2
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
6
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
4
Elements of the B-Tree : 2 3 4 6
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
3
Enter the element to be searched :
5
Element not found in B-Tree
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
3
Enter the element to be searched :
6
Element found in B-Tree
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit

Enter your option :
2
Enter the element to be deleted :
4
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
4
Elements of the B-Tree : 2 3 6
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
5

Test Case - 2
User Output
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
2
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
4
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
4
Elements of the B-Tree : 2 4
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
2
Enter the element to be deleted :
6
Element not found in B-Tree.
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
2
Enter the element to be deleted :
4
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit

Enter your option :
2
Enter the element to be deleted :
2
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
4
B-Tree is empty
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
5

Test Case - 3
User Output
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
33
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
22
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
1
Enter an element to be inserted :
11
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
4
Elements of the B-Tree : 11 22 33
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
3
Enter the element to be searched :
23
Element not found in B-Tree
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit

Enter your option :
3
Enter the element to be searched :
33
Element found in B-Tree
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
2
Enter the element to be deleted :
33
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
3
Enter the element to be searched :
33
Element not found in B-Tree
1.Insert 2.Delete 3.Search 4.Traversal 5.Exit
Enter your option :
5

S.No: 4	Exp. Name: ***Min Heap Operations***	Date: 2026-09-10

Aim:

Write a C program that creates a Min Heap of integers, displays the heap, and performs the deletion (extract-min) operation.

Input Format:

- The program first prompts the user to enter:

`Enter no of elements:`

- Read an integer n, representing the number of elements to be inserted into the Min Heap.
- The program then prompts:

`Enter elements:`

- Read n space-separated integers representing the elements to be inserted into the Min Heap.

Output Format:

- Print the Min Heap after all insertions in the following format:

`Min Heap: <heap_elements>`

- where <heap_elements> are the elements of the Min Heap printed in level-order, separated by spaces.
- Then print:

`Extracting elements from the min heap:`

- For each extraction, print:

`Extracted: <element>`

- where <element> is the minimum element extracted from the Min Heap.
- Each extracted element should be printed on a new line until the heap becomes empty.

Source Code:

minHeapOfIntegers.c

```
// Write your code here...
#include <stdio.h>

#define MAX_HEAP_SIZE 100

// Function to swap two integers
void swap(int *x, int *y) {
        int temp = *x;
        *x = *y;
        *y = temp;
}

// Function to perform min heapify on a subtree rooted at index if
void minHeapify(int arr[], int n, int i) {
        int smallest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        if(left < n && arr[left] < arr[smallest])
                smallest = left;
        if(right < n && arr[right] < arr[smallest])
                smallest = right;
        if(smallest != i){
                swap(&arr[i], &arr[smallest]);
                minHeapify(arr, n, smallest);
        }
}

// Funtion to build a min heap
void buildMinHeap(int arr[], int n){
        for(int i = n/ 2 - 1; i>= 0;i--)
                minHeapify(arr, n ,i);
}

// Funtion to display the min heap
void displayHeap(int arr[], int n){
        printf("Min Heap: ");
        for(int i = 0;i < n;i++)
                printf("%d ",arr[i]);
        printf("\n");
}

//Funtion to perform deletion (extract the minimum element) from the min
heap
int extractMin(int arr[],int *n){
```

```
        if(*n <= 0)
                return -1;
        int minElement = arr[0];
        arr[0] = arr[*n - 1];
        (*n)--;
        if(*n > 0)
                minHeapify(arr,*n,0);
        return minElement;
}

int main(){
        int arr[MAX_HEAP_SIZE];
        int n;

        printf("Enter no of elements: ");
        scanf("%d", &n);

        printf("Enter elements: ");
        for(int i = 0;i < n;i++)
                scanf("%d", &arr[i]);

        buildMinHeap(arr, n);
        displayHeap(arr, n);

        printf("Extracting elements from the min heap:\n");
        while(n > 0){
                int minElement = extractMin(arr, &n);
                if(minElement == -1)
                        printf("Heap is empty.\n");
                else
                        printf("Extracted: %d\n", minElement);
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter no of elements:
5

Enter elements:
1 2 6 4 8
Min Heap: 1 2 6 4 8
Extracting elements from the min heap:
Extracted: 1
Extracted: 2
Extracted: 4
Extracted: 6
Extracted: 8

Test Case - 2
User Output
Enter no of elements:
6
Enter elements:
2 5 1 6 8 4
Min Heap: 1 5 2 6 8 4
Extracting elements from the min heap:
Extracted: 1
Extracted: 2
Extracted: 4
Extracted: 5
Extracted: 6
Extracted: 8

S.No: 5	Exp. Name: ***Max Heap of Integers***	**Date: 2026-09-10**

Aim:

Write a C program that creates a Max Heap of integers, displays the heap, and performs the deletion (extract-max) operation.

Input Format:

- The program first prompts the user to enter:

`Enter no of elements:`

- Read an integer n, representing the number of elements to be inserted into the Max Heap.
- The program then prompts:

`elements:`

- Read n space-separated integers representing the elements to be inserted into the Max Heap.

Output Format:

- Print the Max Heap after all insertions in the following format:

`Max Heap: <heap_elements>`

- where <heap_elements> are the elements of the Max Heap printed in level-order, separated by spaces.
- Then print:

`Extracting elements from max heap:`

- For each extraction, print:

`Extracted: <element>`

- where <element> is the maximum element extracted from the Max Heap.
- Each extracted element should be printed on a new line until the heap becomes empty. The extracted elements will be printed in descending order.

Source Code:

maxHeapOfIntegers.c

```
// Write your code here...
#include <stdio.h>

#define MAX_HEAP_SIZE 100

// Funtion to swap two integers
void swap(int *x, int *y){
        int temp = *x;
        *x = *y;
        *y = temp;
}

// Function to perfrom max heapify on a subtree rooted at indexx i
void maxHeapify(int arr[], int n, int i){
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        if(left < n && arr[left] > arr[largest])
                largest = left;
        if(right < n && arr[right] > arr[largest])
                largest = right;
        if(largest != i){
                swap(&arr[i],&arr[largest]);
                maxHeapify(arr,n,largest);
        }
}

// Function to build a max heap
void buildMaxHeap(int arr[], int n){
        for(int i = n / 2 - 1;i >= 0;i--){
                maxHeapify(arr,n,i);
        }
}

// Function to display the max heap
void displayHeap(int arr[], int n){
        printf("Max Heap: ");
        for(int i = 0;i < n;i++){
                printf("%d ",arr[i]);
        }
        printf("\n");
}

// Function to perform deletion (extract the maximum element) from the
```

```
max heap
int extractMax(int arr[], int*n) {
        if(*n <= 0)
                return -1;
        int max = arr[0];
        arr[0] = arr[*n - 1];
        (*n)--;
        maxHeapify(arr, *n, 0);

        return max;
}

int main() {
        int arr[MAX_HEAP_SIZE];
        int n;

        printf("Enter no of elements: ");
        scanf("%d", &n);

        printf("elements: ");
        for(int i = 0;i < n;i++)
                scanf("%d", &arr[i]);

        buildMaxHeap(arr, n);
        displayHeap(arr, n);

        printf("Extracting elements from max heap:\n");
        while(n > 0){
                int maxElement = extractMax(arr, &n);
                if(maxElement == -1)
                        printf("Heap is empty.\n");
                else
                        printf("Extracted: %d\n", maxElement);
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

Enter no of elements:
5
elements:
6 4 5 8 2
Max Heap: 8 6 5 4 2
Extracting elements from max heap:
Extracted: 8
Extracted: 6
Extracted: 5
Extracted: 4
Extracted: 2

Test Case - 2
User Output
Enter no of elements:
6
elements:
1 5 2 9 8 4
Max Heap: 9 8 4 5 1 2
Extracting elements from max heap:
Extracted: 9
Extracted: 8
Extracted: 5
Extracted: 4
Extracted: 2
Extracted: 1

S.No: 6	Exp. Name: ***Bi-Connected Components in Graph***	**Date: 2026-09-10**

Aim:

Write a C program to find the number of biconnected components (BCCs) in a given undirected graph.

Input Format:

- The first line contains a single integer, V, the number of vertices in the graph.
- The second line contains a single integer, E, the number of edges.
- Each of the next E lines contains two space-separated integers u and v, representing an edge between vertex u and vertex v.

Output Format:

- Print a single integer: the number of biconnected components in the graph.

Constraints:

- $1 \leq V \leq 1000$
- $0 \leq E \leq V * (V-1)/2$
- $0 \leq u, v < V$

Source Code:

biConnected.c

```
// Write your code here...
#include <stdio.h>
#include <stdlib.h>

#define MAXV 1000
#define MAXE 500000

struct Edge{
int to;
int id;
struct Edge *next;
};

struct Edge *adj[MAXV];

int disc[MAXV];
int low[MAXV];

int timeCounter = 0;
int bccCount = 0;

int edgeStack[MAXE];
int top = -1;

void push(int id){
        edgeStack[++top] = id;
}
int pop(){
        return edgeStack[top--];
}

void addEdge(int u,int v,int id){
        struct Edge *newEdge;

        newEdge = (struct Edge*)malloc(sizeof(struct Edge));

        newEdge->to = v;
        newEdge->id = id;
        newEdge->next = adj[u];

        adj[u] = newEdge;
}
void DFS(int u,int parentEdge){
        struct Edge*p;
```

```
        int v;
        int id;

        disc[u] = low[u] = ++timeCounter;
        p = adj[u];

        while(p != NULL){
                v = p->to;
                id = p->id;

                /*ignore the edge used to enter this vertex*/
                if(id == parentEdge){
                        p = p->next;
                        continue;
                }
        /*Tree edge*/
                if(disc[v] == 0){
                        push(id);

                        DFS(v,id);

                        if(low[v] < low[u])
                                low[u] = low[v];

                /*BCC is formed when this condition is true.*/
                        if(low[v] >= disc[u]){
                                do{
                                        id = pop();
                                } while(id != p->id);

                                bccCount++;
                        }
                }
        /*Back edge*/
                else if(disc[v] < disc[u]){
                        push(id);

                if(disc[v] < low[u])
                        low[u] =disc[v];
                }

                p = p -> next;
        }
}

int main(){
```

```
        int V,E;
        int u,v;
        int i;

        scanf("%d", &V);
        scanf("%d", &E);

        for(i = 0;i < V;i++){
                adj[i] = NULL;
                disc[i] = 0;
                low[i] = 0;
        }

        for(i = 0;i < E;i++){
                scanf("%d %d", &u, &v);

                addEdge(u,v,i);
                addEdge(v,u,i);
        }

        timeCounter = 0;
        bccCount = 0;
        top = -1;

        for (i = 0;i < V;i++){
                if(disc[i] == 0){
                        DFS(i,-1);
                }
        }

        printf("%d\n",bccCount);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
2
0 1

0 2
2

Test Case - 2
User Output
5
4
0 1
1 2
1 3
2 4
4

S.No: 7	Exp. Name: ***Merge Sort***	**Date: 2026-09-10**

Aim:

Write a C program to implement the Merge Sort algorithm to sort an array of integers in ascending order.

Input Format:

- The first line of input prompts the user to enter the array size (n) as "Enter array size: ".
- The second line of input reads n space-separated integers after displaying "Enter <n> elements: ".

Output Format:

- The first line of output prints "Before sorting: " followed by the original array elements.
- The second line of output prints "After sorting: " followed by the sorted array in ascending order.

Source Code:

mergeSort.c

```
// Write your code here...
#include <stdio.h>
void display(int arr[15], int n){
        int i;
        for(i = 0;i < n;i++)
                printf("%d ",arr[i]);
        printf("\n");
}
void merge(int arr[15], int low, int mid, int high){
        int i = low,h = low,j = mid + 1,k,temp[15];
        while(h <= mid && j <= high){
                if(arr[h] <= arr[j]){
                        temp[i] = arr[h];
                        h++;
                }else{
                        temp[i] = arr[j];
                        j++;
                }
                i++;
        }
        if(h > mid){
                for(k = j;k <= high;k++){
                        temp[i] = arr[k];
                        i++;
                }
        }else{
                for(k = h;k <= mid;k++){
                        temp[i] = arr[k];
                        i++;
                }
        }
        for(k = low;k <= high;k++){
                arr[k] = temp[k];
        }
}
void splitAndMerge(int arr[15],int low,int high){
        int mid;
        if(low < high){
                mid = (low + high)/2;
                splitAndMerge(arr,low,mid);
                splitAndMerge(arr,mid+1,high);
                merge(arr,low,mid,high);
        }
}
```

```
void main() {
        int arr[15], i, n;
        printf("Enter array size: ");
        scanf("%d",&n);
        printf("Enter %d elements: ", n);
        for(i = 0;i < n;i++){
                scanf("%d", &arr[i]);
        }
        printf("Before sorting: ");
        display(arr, n);
        splitAndMerge(arr,0,n - 1);
        printf("After sorting: ");
        display(arr, n);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter array size:
5
Enter 5 elements:
1 2 3 5 4
Before sorting: 1 2 3 5 4
After sorting: 1 2 3 4 5

Test Case - 2
User Output
Enter array size:
8
Enter 8 elements:
95 14 10 2 32 5 9 4
Before sorting: 95 14 10 2 32 5 9 4
After sorting: 2 4 5 9 10 14 32 95

S.No: 8	Exp. Name: ***Quick Sort***	**Date: 2026-09-10**

Aim:

Write a C program that implements the Quick Sort to sort a given list of integers in ascending order.

Input Format

- The program should prompt the user to enter the size of the array and the elements of the array.

Output Format

- After sorting the array, the program should display the sorted array.

Source Code:

quickSort.c

```
// Write your code here...
#include <stdio.h>

void display(int arr[15], int n){
        int i;
        for(i = 0;i < n;i++)
                printf("%d ", arr[i]);
        printf("\n");
}
int partition(int arr[15], int lb, int ub ){
        int pivot,start,end,temp;
        pivot = arr[lb];
        start = lb;
        end = ub;
        while(start < end){
                while(arr[start] <= pivot && start < ub)
                        start ++;
                while(arr[end] > pivot)
                        end--;
                if(start < end){
                        temp = arr[start];
                        arr[start] = arr[end];
                        arr[end] = temp;
                }
        }
        temp = arr[lb];
        arr[lb] = arr[end];
        arr[end] = temp;

        return end;
}
void quickSort(int arr[15], int low,int high){
        int j;
        if(low < high){
                j = partition(arr,low,high);
                quickSort(arr,low,j - 1);
                quickSort(arr,j + 1,high);
        }
}

void main() {
        int arr[15], i ,n;
        printf("Enter array size : ");
        scanf("%d", &n);
```

```
        printf("Enter %d elements : ",n);
        for(i = 0;i < n;i++){
                scanf("%d", &arr[i]);
        }
        printf("Before sorting the elements are : ");
        display(arr, n);
        quickSort(arr, 0 ,n - 1);
        printf("After sorting the elements are : ");
        display(arr, n);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter array size :
6
Enter 6 elements :
6 2 8 10 36 14
Before sorting the elements are : 6 2 8 10 36 14
After sorting the elements are : 2 6 8 10 14 36

Test Case - 2
User Output
Enter array size :
8
Enter 8 elements :
95 14 10 23 36 2 5 35
Before sorting the elements are : 95 14 10 23 36 2 5 35
After sorting the elements are : 2 5 10 14 23 35 36 95

S.No: 9	Exp. Name: ***Shortest Path from source to vertex***	**Date: 2026-09-26**

Aim:

You are required to write a C program that computes the shortest paths from a source vertex to all other vertices in a weighted, directed graph. The program should utilize Dijkstra's algorithm to solve this problem.

The program must perform the following tasks:

Input the Number of Vertices:

- Print: "Enter the number of vertices: "
- Read an integer V representing the number of vertices in the graph.

Input the Adjacency Matrix:

- Print: "adjacency matrix:"
- For each pair of vertices (i, j), print: "Weight from i to j: "
- Read the weight of the edge between vertex i and vertex j. If there is no edge between them, input 0.

Input the Source Vertex:

- Print: "Enter the source vertex: "
- Read the source vertex src.

Validation:

- Ensure that the number of vertices does not exceed 100. If it does, print: "Number of vertices exceeds maximum allowed (100)" and exit the program.
- Ensure that the source vertex is within the valid range (0 to V-1). If it is not, print: "Invalid source vertex" and exit the program.

Compute Shortest Paths:

- Use Dijkstra's algorithm to compute the shortest paths from the source vertex to all other vertices.

Note: Refer to sample test cases for better understanding

Source Code:

Path.c

```
// Write your code here...
#include <stdio.h>
#define MAX 100
#define INF 99999

int main(){
        int V, src;
        int graph[MAX][MAX];
        int dist[MAX];
        int visited[MAX];
        int i, j, count, u, min;
        printf("number of vertices: ");
        scanf("%d", &V);
        if(V > 100){
                printf("Number of vertices exceeddsmaximum
allowed(100)\n");
                return 0;
        }
        printf("adjacency matrix:\n");
        for(i = 0;i < V;i++){
                for(j = 0;j < V;j++){
                        printf("Weight from %d to %d: ", i, j);
                        scanf("%d", &graph[i][j]);
                }
        }
        printf("Enter the source vertex: ");
        scanf("%d", &src);
        if(src < 0 || src >= V){
                printf("Invalid source vertex\n");
                return 0;
        }
        for(i = 0;i < V;i++){
                dist[i] = INF;
                visited[i] = 0;
        }
        dist[src] = 0;
        //Dijkstra's algorithm
        for(count = 0;count < V-1;count++){
                min = INF;
                u = -1;
                for(i = 0;i < V;i++){
                        if(!visited[i] && dist[i] < min){
                                min = dist[i];
                                u = i;
```

```
                        }
                }
                if( u == -1){
                        break;
                }
                visited[u] = 1;
                for(j = 0;j < V;j++){
                        if(visited[j] == 0 && graph[u][j] != 0 &&
dist[u] + graph[u][j] < dist[j]){
                                dist[j] = dist[u] + graph[u][j];
                        }
                }
        }
        printf("Vertex \t Distance from Source\n");
        for(i = 0;i < V;i++){
                printf("%d \t %d\n", i, dist[i]);
        }
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
number of vertices:
3
adjacency matrix:
Weight from 0 to 0:
0
Weight from 0 to 1:
5
Weight from 0 to 2:
10
Weight from 1 to 0:
5
Weight from 1 to 1:
0
Weight from 1 to 2:
2
Weight from 2 to 0:

10
Weight from 2 to 1:
2
Weight from 2 to 2:
0
Enter the source vertex:
0
Vertex Distance from Source
0 0
1 5
2 7

S.No: 10	Exp. Name: ***Job Sequencing using Greedy Problem***	**Date: 2026-09-17**

Aim:

You are given a set of N jobs, where each job i has:

- a deadline
- a profit

Each job takes 1 unit of time to complete. Only one job can be scheduled at a given time slot. You earn the profit of a job only if it is completed on or before its deadline.

Your task is to determine:

- The maximum number of jobs that can be completed, and
- The maximum profit that can be earned.

Input Format:

- First line: An integer N representing the number of jobs.
- Second line: A sequence of $3N$ integers, where each set of 3 integers represents a job in the format:

```
JobID Deadline Profit
```

Output Format

A single line output prints two integers separated by a space:

- The first integer represents the number of jobs done.
- The second integer represents the maximum profit obtained.

Example 1:

Input:

```
N = 4
Jobs = {(1,4,20),(2,1,10),(3,1,40),(4,1,30)}
```

Output:

```
2 60
```

Explanation: Job 1 and Job 3 can be scheduled to achieve a maximum profit of 20 + 40 = 60.

Example 2:

Input:

```
N = 5
Jobs = {(1,2,100),(2,1,19),(3,2,27),(4,1,25),(5,1,15)}
```

Output:

```
2 127
```

Explanation: Two jobs (Job 1 and Job 3) can be completed for a maximum profit of 100 + 27 = 127.

Constraints:

- $1 \leq N \leq 10^5$

- $1 \leq Deadline \leq N$
- $1 \leq Profit \leq 500$

Note:

- Jobs will be given in the form (Jobid, Deadline, Profit)
- A job must be completed on or before its deadline in order to earn the associated profit.

Source Code:

JsGreedy.c

```
// Write your code here...
#include <stdio.h>
#include <stdlib.h>

struct Job{
int id;
int deadline;
int profit;
};

// Sort jobs in decreasing order of profit
int compare(const void *a,const void *b){
        struct Job *j1 = (struct Job*)a;
        struct Job *j2 = (struct Job*)b;

        return j2 -> profit - j1 -> profit;
}

// find the latest available slot
int find(int parent[], int x){
        if(parent[x] == x)
                return x;

        parent[x] = find(parent,parent[x]);
        return parent[x];
}

int main() {
        int n;
        scanf("%d",&n);
        struct Job jobs[n];
        int maxDeadline = 0;

        //read JobID,Deadline,profit
        for(int i = 0;i < n;i++){

scanf("%d%d%d",&jobs[i].id,&jobs[i].deadline,&jobs[i].profit);

                if(jobs[i].deadline > maxDeadline)
                        maxDeadline = jobs[i].deadline;
        }

        //sort according to decreasing profit
        qsort(jobs,n,sizeof(struct Job),compare);
```

```
        //parent[i] tells the latest free slot available at or before
slot i
        int parent[maxDeadline + 1];
        for (int i = 0;i <= maxDeadline;i++)
                parent[i] = i;
        int count = 0;
        int totalProfit = 0;
        //schedule each job
        for(int i = 0;i < n;i++){
                int slot = find(parent,jobs[i].deadline);
                if(slot > 0){
                        //schedule the jobs
                        count++;
                        totalProfit += jobs[i].profit;

                        //mark this slot as occupied
                        parent[slot] = find(parent,slot - 1);
                }
        }

        printf("%d %d\n",count,totalProfit);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
1 4 20 2 1 10 3 1 40 4 1 30
2 60

Test Case - 2
User Output
5
1 2 100 2 1 19 3 2 27 4 1 25 5 1 15
2 127

S.No: 11	Exp. Name: ***Problem on N Queens***	Date: 2026-10-01

Aim:

Write a program to arrange N queens on a $N \times N$ chessboard using backtracking algorithm, such that no queen can attack any other queens on the board, and print the result as shown in the example.

Input Format:

- A single line of integer representing the size of the board and number of queens.

Output Format:

- An $N \times N$ matrix where each element is either 0 or 1.
- Each row is printed on a new line with tab-separated values.

Sample Input and Output:

```
Enter the value of N for NxN chessboard: 4
0	1	0	0
0	0	0	1
1	0	0	0
0	0	1	0
```

Source Code:

backtrack.c

```
// Write your code here...
#include <stdio.h>
#include <stdlib.h>
int board[20][20];
int num;
int isSafe(int row, int col){
        int i,j;
        for(i = 0;i < row;i++){
                if(board[i][col] == 1){
                        return 0;
                }
        }
        i = row - 1;
        j = col - 1;
        while(i >= 0 && j >= 0){
                if(board[i][j] == 1)
                        return 0;
        i--;
        j--;
        }
        i = row - 1;
        j = col + 1;
        while(i >= 0 && j < num){
                if(board[i][j] == 1)
                        return 0;
                i--;
                j++;
        }
        return 1;
}
int solve(int row){
        int col;
        if(row == num)
                return 1;
        for(col = 0 ;col < num;col++){
                if(isSafe(row,col)){
                        board[row][col] = 1;
                        if(solve(row + 1))
                                return 1;
                        board[row][col] = 0;
                }
        }
        return 0;
}
```

```
int main(){
        int i,j;
        printf("Enter the value of N for NxN chessboard: ");
        scanf("%d",&num);
        if(solve(0)){
                for(i = 0;i < num;i++){
                        for(j = 0;j < num;j++){
                                printf("%d",board[i][j]);
                                printf("\t");
                        }
                        printf("\n");
                }
        }
        else{
                printf("No solution exists\n");
        }
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1			
User Output			
Enter the value of N for NxN chessboard:			
4			
0	1	0	0
0	0	0	1
1	0	0	0
0	0	1	0

Test Case - 2							
User Output							
Enter the value of N for NxN chessboard:							
8							
1	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	1
0	0	0	0	0	1	0	0

0	0	1	0	0	0	0	0
0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0
0	0	0	1	0	0	0	0

S.No: 12	Exp. Name: ***Knapsack using backtracking***	**Date: 2026-09-27**

Aim:

Write a C program to solve the 0/1 Knapsack Problem using backtracking approach. The program aims to maximize the total value of items that can be included in a knapsack without exceeding its capacity

Input Format:

- Prompt the user to enter an integer n, indicating the number of items.
- For each item, prompt the user to enter its value and weight separated by space.
- Prompt the user to enter the capacity of the knapsack.

Output Format:

- Print the maximum profit (total value) that can be obtained by selecting items, considering the knapsack's capacity constraint.

Source Code:

knapsackBacktracking.c

```
// Write your code here...
#include <stdio.h>
int n;
int weight[100],value[100];
int w;
int maxProfit = 0;
void knapsack(int i,int currentWeight,int currentProfit){
        if(i == n){
                if(currentProfit > maxProfit)
                        maxProfit = currentProfit;
                return;
        }
        if(currentWeight + weight[i] <= w){
                knapsack(i+1,currentWeight + weight[i],currentProfit +
value[i]);
        }
        knapsack(i + 1,currentWeight,currentProfit);
}
int main(){
        int i;
        scanf("%d",&n);
        for(i = 0;i < n;i++){
                scanf("%d %d", &value[i],&weight[i]);
        }
        scanf("%d",&w);
        knapsack(0, 0, 0);
        printf("%d\n", maxProfit);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
60 10
100 20
120 30
50

220

Test Case - 2
User Output
5
10 5
40 4
30 3
50 2
20 1
100
150

S.No: 13	Exp. Name: ***Program to implement Knapsack problem Using Dynamic Programming***	**Date: 2026-09-26**

Aim:

You are given n items, each with a specific weight and value. You have a knapsack with a maximum capacity W. Your goal is to select a subset of these items such that the total weight is less than or equal to W, and the total value is as large as possible.

This is the classic 0/1 Knapsack problem, where each item can either be included once or excluded from the knapsack.

Input Format:

- Prompt the user to enter the number of items n to consider for the knapsack.
- Prompt the user to enter the weights of these n items, in a single line separated by a space.
- Prompt the user to enter the values of these n items, corresponding to their weights in a single line separated by a space.
- Prompt the user to enter the knapsack capacity W.

Output Format:

- Print the maximum profit that can be obtained by selecting items such that their combined weight does not exceed the knapsack capacity.

Source Code:

dynamicKs.c

```
// Write your code here...
#include <stdio.h>
int n;
int weight[100], value[100];
int W;
int maxProfit = 0;
void knapsack(int i, int currentWeight, int currentProfit){
        if( i == n){
                if(currentProfit > maxProfit)
                        maxProfit = currentProfit;
                return;
        }
        if(currentWeight + weight[i] <= W){
                knapsack(i + 1, currentWeight + weight[i], currentProfit
+ value[i]);
        }
        knapsack(i + 1, currentWeight, currentProfit);
}
int main(){
        int i;
        scanf("%d", &n);
        for(i = 0;i < n;i++)
                scanf("%d", &weight[i]);
        for(i = 0;i < n;i++)
                scanf("%d", &value[i]);
        scanf("%d", &W);
        knapsack(0, 0, 0);
        printf("%d\n", maxProfit);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
5 10 15 20 25
10 20 30 40 50
500

150

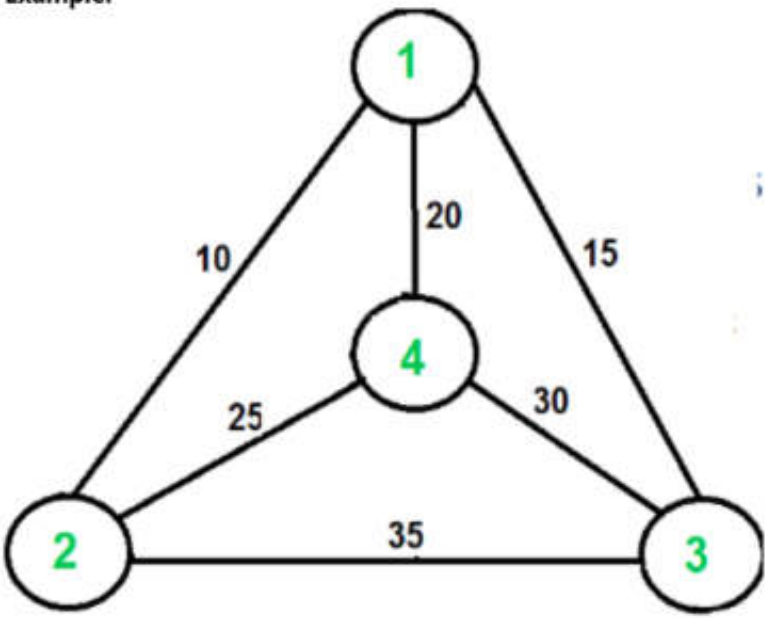
Test Case - 2
User Output
4
100 200 300 400
10 20 30 40
100
10

S.No: 14	Exp. Name: ***Travelling Salesman Problem***	**Date: 2026-10-01**

Aim:

Given a set of cities and the distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

Example:



A TSP tour in the graph is 1-2-4-3-1. The cost of the tour is 10+25+30+15, which is 80

Input Format:

- The first line contains an integer n, representing the number of cities.
- The following n lines contain the cost matrix of size $n \times n$. Each line contains n integers separated by spaces, representing the cost of traveling from one city to another. If there is no direct connection between two cities, -1 is used to represent infinity.

Output Format:

- The output is a single integer representing the optimal cost of the traveling salesman problem.

Source Code:

CTC30966.c

```
#include <stdio.h>
int cost[20][20];
int visited[20];
int num;
int minCost = 999999;
void tsp(int city,int count,int totalCost){
        int next;
        if(count == num){
                if(cost[city][0] != -1){
                        totalCost = totalCost + cost[city][0];
                        if(totalCost < minCost)
                                minCost = totalCost;
                }
                return;
        }
        for(next = 0;next < num;next++){
                if(visited[next] == 0 && cost[city][next] != -1){
                        visited[next] = 1;
                        tsp(next,count + 1,totalCost + cost[city]
[next]);
                        visited[next] = 0;
                }
        }
}
int main(){
        int i,j;
        scanf("%d",&num);
        for(i = 0;i < num;i++){
                for(j = 0;j < num;j++){
                        scanf("%d",&cost[i][j]);
                }
        }
        for(i = 0;i < num;i++)
                visited[i] = 0;
        visited[0] = 1;
        tsp(0,1,0);
        printf("%d\n",minCost);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Ramachandra College of Engineering 2025-2029-AIDS-B ID: 25ME1A5483 Page No: 71

Test Case - 1
User Output
3
-1 2 3
4 -1 6
1 2 -2
9

Test Case - 2
User Output
4
-1 2 3 4
1 -1 1 3
4 5 -1 1
2 3 6 -1
6